Lecture 1: Introduction to Cloud Computing

IT4090 Cloud Computing

- Evolution of IT infrastructure
- NIST definition and 5 characteristics
- Service models and deployment models
- Cloud providers and benefits

Evolution of IT infrastructure

Five stages, from central machines to services on demand

- **Mainframe / Mini** computers
- **Personal computer**
- **Client / server** computing
- **Web-based** enterprise applications
- **Cloud** and mobile computing

The electricity analogy

Cloud is to IT what the power grid is to electricity

- Mid 1800s: factories made their own power (dynamos): costly, needed engineers, not the core business
- Late 1800s: power companies sold electricity over wires
- Customers just **plug in and pay for what they use**
- Cloud is the same idea for IT: no own data center, focus on core business

Traditional IT vs cloud

	Traditional IT	Cloud
Time to go live	6 to 12 months (vendors, PO, hardware, software)	Minutes
Cost model	Big upfront **CAPEX**	Pay as you go **OPEX**
Capacity	Fixed: under-supply or wasted capacity	Add or remove as the load changes
Innovation	Barrier: slow and costly to try ideas	Cheap to try, destroy when done

Why do we need cloud?

- **Business agility**: faster time to market, flexibility and scalability
- **Customer experience**: always available services, many regions
- **Cost**: pay as you go, CAPEX to OPEX, focus on core business

NIST definition of cloud computing

Say this definition in the exam

- A model for **ubiquitous, convenient, on-demand network access**
- to a **shared pool of configurable resources** (networks, servers, storage, apps, services)
- that can be **rapidly provisioned and released** with minimal management effort

5 essential characteristics

Trick: B-R-R-O-M

Characteristic	Meaning
On-demand self-service	Provision resources yourself, no provider contact
Broad network access	Available over the network from any standard client
Resource pooling	Shared pool, multi-tenant, location only at region or country level
Rapid elasticity	Scale out and in quickly, often automatically
Measured service	Metering, pay per use, monitor and report

Is cloud a technology or a model?

- Cloud is **not** a technology
- It is a **business model** and a **delivery model**
- The technology underneath is **virtualization**

Service models

Model	Provider manages	You manage	Examples
IaaS	Physical infrastructure and virtualization	Servers, OS, platform, apps	AWS EC2, Azure VM, S3
PaaS	Infrastructure and platform	Your application and its settings	Elastic Beanstalk, Azure App Service, RDS
SaaS	Everything including the app	Only user settings	Gmail, Microsoft 365, Salesforce

Actors in cloud computing (NIST)

Actor	Role
Cloud consumer	Uses services from a provider
Cloud provider	Makes the service available
Cloud auditor	Independent assessment of services, security, performance
Cloud broker	Manages use and delivery, negotiates between provider and consumer
Cloud carrier	Connectivity and transport from provider to consumer

Deployment models

Model	Key points
Public	Open to everyone, provider site, multi-tenant. Cheap, scalable, zero maintenance. Lower data security
Private	One organization, single tenant. High security and control. Expensive, management overhead
Hybrid	Private plus public, data and apps portable between them. Flexible. Data transfer is the issue
Community	Shared by a community with the same concerns, e.g. **government clouds**

Cloud service providers

- About 300 providers worldwide
- Top 6: **AWS, Azure, Google Cloud, IBM Cloud, Alibaba Cloud, Oracle Cloud**
- Top 3 hold about 60%: AWS about 33%, Azure about 20%, GCP about 7%

Lecture 2: Core Services and Global Infrastructure

IT4090 Cloud Computing

- Region, Availability Zone, data center
- Choosing a Region
- Core services
- Shared responsibility, billing, quotas

Cloud global infrastructure

- Hardware and software that make up the cloud: servers, storage, networking in data centers worldwide
- Designed to be **flexible, reliable, scalable and secure**
- Benefits: **security, availability, performance, scalability, flexibility**

Region, Availability Zone, data center

Region

a geographic area

has several AZs

Availability Zone

fully isolated partition

one or more data centers

joined by fast private network

Data center

where data lives and is processed

redundant power and networking

separate facility

Choosing a Region

Trick: Legal, Latency, Services, Cost

Factor	Why
Data governance and legal rules	Some data may not leave the country
Proximity to customers	Lower latency
Services available	Not every service exists in every Region
Cost	Prices differ by Region

Core cloud services

Category	AWS	Azure
Compute	EC2	Virtual Machines
Network	VPC	Virtual Network
Storage	S3, EBS, EFS	Blob, Disks, Files

Shared responsibility model

More managed service = less for you to secure

Provider

Security **OF** the cloud

hardware, data centers, network, virtualization

Customer

Security **IN** the cloud

data, access (IAM), OS patches on IaaS, firewall rules, encryption

How you pay

Pay for what you use
no large upfront cost

Pay less when you reserve
Reserved Instances

Pay less when you use more
volume discounts

Service limits and quotas

- Default quotas, **specific to each Region**
- **Soft limit**: can be increased on request
- **Hard limit**: cannot be increased
- Why: prevent excessive charges and overuse by one customer

Lecture 3: Compute and Storage

IT4090 Cloud Computing

- Virtualization vs containers
- EC2 and instance types
- Purchasing options and lifecycle
- Storage: block, file, object, CAP

Virtualization vs containerization

Type 1 hypervisor runs on hardware (cloud). Type 2 runs on a host OS

Virtual machine

own guest OS each

hypervisor on hardware

heavy, slower start

Container

shares host OS kernel

container engine (Docker)

light, starts in seconds

Software defined compute

- Compute function is **virtualized and abstracted** from hardware
- Processing power is **pooled** across many units and workload is spread
- Generic, industry standard hardware, easy to add

Cloud compute services

	Amazon EC2	Azure VM
Model	IaaS	IaaS
Template	AMI	Marketplace image
Billing	Per second	Per minute

Instance types

Type	Use
General purpose	Balanced: web servers, small databases
Compute optimized	High CPU: batch, gaming, modelling
Memory optimized	High RAM: in-memory databases
Storage optimized	High disk I/O: warehouses, NoSQL
Accelerated computing	GPU: machine learning, graphics
Burstable	Low baseline, bursts when needed

Tenancy

Option	Meaning
Default / shared	Same hardware shared with other customers
Dedicated instances	Hardware used only by your account
Dedicated hosts	Whole physical server, good for per-socket licences

Purchasing options

Option	Key point	Best for
On-demand	No commitment, highest price	Short, unpredictable work
Reserved	1 or 3 year commitment, big discount	Steady workloads
Spot	Cheapest, can be **stopped by provider**	Fault-tolerant batch jobs
Savings plan	Commit $ per hour, flexible instance type	Changing but steady usage

Instance lifecycle

- Pending, then **Running**
- **Stop**: shut down, can start again, pay for storage only
- **Hibernate**: save memory to disk, resume later
- **Terminate**: deleted for good, cannot return

Placement groups

Azure equivalent: Proximity placement group

Type	Purpose
Cluster	Same AZ, close together: lowest latency, high throughput
Partition	Groups on separate racks: big distributed systems (Hadoop, Cassandra)
Spread	Each instance on different hardware: maximum availability

Software defined storage and CAP theorem

- **Software defined storage**: storage abstracted from the hardware
- **CAP**: a distributed store can give only **2 of 3**: **C**onsistency, **A**vailability, **P**artition tolerance
- During a network partition choose: cancel the operation (consistency) or carry on (availability, risk of stale data)

Block, file and object storage

	Block	File	Object
Structure	Blocks with IDs	Folders and files	Data + metadata + unique ID
Access	Attached to a VM	NFS / SMB share	HTTPS, API, SDK
Best for	Databases, boot disks (IOPS)	Shared content	Backups, media, static websites
AWS	EBS	EFS	S3
Azure	Disks	Files	Blob

Object storage features and tiers

Features	Tiers (AWS / Azure)
Encryption: client side or server side (service, customer managed or customer provided keys) Versioning Immutability: time-based retention, legal hold Static web hosting	S3 Standard / **Hot** Intelligent-Tiering Infrequent Access / **Cold** Glacier / **Archive**

Other storage topics

- **Archiving and backup**
- **Hybrid storage**: on-premises plus cloud
- **Bulk data transfer** with physical devices (Snowball, Data Box)

Lecture 4: Cloud Networking

IT4090 Cloud Computing

- SDN and NFV
- VPC, subnets and IP sizing
- Security groups and NACLs
- Gateways, peering, hybrid, private access

SDN and NFV

SDN

separates network **control** from **forwarding**

control becomes programmable

the big picture

NFV

network functions (firewall, router, load balancer)

run as software in VMs

VNF, and CNF in containers

specific functions

Virtual networks

- **AWS VPC** / **Azure VNet**
- Logically isolated, one or more IP ranges and subnets
- Can peer with other networks if IP ranges **do not overlap**
- Subnet CIDR blocks cannot overlap inside a network

Subnets: public and private (AWS)

- **Public subnet**: route to the internet gateway, instances can send traffic out directly
- **Private subnet**: no direct internet route
- Azure has no public/private concept, use route tables
- Allowed sizes: AWS /16 to /28, Azure /2 to /29

Subnet IP maths (AWS)

First usable IP = network address + 4

- Private ranges: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16
- **5 IPs are reserved** in every subnet: network address, router, DNS, future use, broadcast
- **Usable IPs = 2^(32 - prefix) - 5**
- 10.0.0.0/24 = 251 · /25 = 123 · /27 = 27 · /28 = 11

Worked examples

Subnet	First usable	Last usable	Usable IPs
10.0.0.0/24	10.0.0.4	10.0.0.254	251
172.16.10.128/25	172.16.10.132	172.16.10.254	123
192.168.0.64/27	192.168.0.68	192.168.0.94	27

Security group vs NACL vs NSG

Stateful: reply allowed automatically. Stateless: write rules for request and reply

	Security group	NACL	Azure NSG
Applied to	Instance (ENI)	Subnet	Subnet and NIC
State	**Stateful**	**Stateless**	**Stateful**
Rules	Allow only	Allow and deny	Allow and deny

Network interfaces

- Virtual NIC attached to a compute instance
- One public IP and one or more private IPs
- More than one interface per instance is possible
- AWS **ENI**, Azure **NIC**

Gateway services

Gateway	Job
Internet gateway	Connects the network to the internet, scales horizontally, no bandwidth limit (not a separate item in Azure)
NAT gateway	Private instances reach the internet, no inbound connections. Lives in a **public subnet**
VPN gateway	Encrypted tunnel over the internet to on-premises
Transit gateway	Hub to connect many networks (hub and spoke). Azure Virtual WAN

VPN types

- **Site-to-site**: network to network
- **Point-to-site**: one PC to a network
- Azure VPN Gateway does both
- AWS Virtual Private Gateway is site-to-site only, **AWS Client VPN** is for point-to-site

Network peering and bastion host

- Peering links two networks, across accounts too
- **Not transitive**: A-B and B-C does not mean A-C
- **Bastion (jump) host**: in a public subnet with a public IP, used to manage private instances safely

Hybrid connectivity

Need	Options
Remote users	Internet access, Azure point-to-site VPN, AWS Client VPN
Remote networks	Site-to-site VPN (over internet), or **AWS Direct Connect / Azure ExpressRoute** (private dedicated line)

Private access and flow logs

- **Private Link / Private Endpoint**: reach PaaS services through a private IP, not the public internet
- **Azure Service Endpoint**: stays on Azure backbone but still hits the service public endpoint
- **Flow logs** record IP traffic, collected outside the traffic path (no latency effect). Uses: troubleshooting, intrusion detection, compliance

Lecture 5: High Availability and Disaster Recovery

IT4090 Cloud Computing

- Downtime and availability
- Redundancy, failover, replication
- Load balancing
- BCP, DR, backups, RTO and RPO

Availability and downtime

Planned downtime: maintenance. Unplanned: failure or attack

Availability	Downtime per year
99%	3.65 days
99.9%	8.77 hours
99.99%	52.6 minutes
99.999%	5.26 minutes

High availability (HA)

- System keeps an agreed level of uptime for longer than normal
- **Eliminate single points of failure**
- **Reliable crossover**: switch over without losing data
- **Detect failures** as they happen

Redundancy

Type	Meaning
Passive	Extra capacity built in to absorb a failure
Active	System detects failure and reconfigures itself automatically (voting scheme)

Failover and failback

Failover
switch to a standby when the main system fails
detected with a **heartbeat** signal

Failback
return the repaired system to normal service

Replication and HA clusters

- **Active replication**: every replica processes the request
- **Passive replication**: one replica processes, result copied to the others
- Cluster modes: **Active/Active** (share the load) and **Active/Passive** (standby)

Load balancing algorithms

Session persistence: one client sticks to one backend server

Algorithm	How it chooses
Round robin	Servers in turn
Least connections	Server with fewest active connections
Least time	Fastest responding server
IP hash	Same client IP goes to the same server

Business continuity and disaster recovery

- **Business continuity**: keep delivering at acceptable levels after a disruption
- **BCP**: plan with business impact analysis and threat analysis
- **Disaster recovery**: part of BCP, recovers IT systems after a disaster
- DR plan: team, risk evaluation, critical assets, backup strategy, testing

Backup methods

Method	Copies	Restore needs
Full	Everything	One backup
Incremental	Changes since the last backup	Full + all incrementals
Differential	Changes since the last full	Full + latest differential
Near-continuous (CDP)	Almost every change	Almost no data loss

DR tiers

- Tier 0: no off-site data
- Tier 1: backup only, no hot site
- Tier 4: backup with hot site
- Tier 5: electronic vaulting · Tier 6: point-in-time copies
- **Tier 7**: transaction integrity, zero or little data loss, highly automated

RTO, RPO, RCO

RPO of 1 hour means back up at least every hour

Term	Meaning
RTO	Time within which a process must be restored
RPO	How much data loss (in time) is acceptable
RCO	How much data inconsistency is acceptable after recovery

Lecture 6: Security and Monitoring

IT4090 Cloud Computing

- Software defined security and PLOP
- IAM, MFA, federation and SSO
- Encryption
- Risks and monitoring

Software defined security and PLOP

- **Software defined security**: controls (intrusion detection, segmentation, access) are policy-driven, automated and run by software
- **Principle of Least Privilege**: give a user or process only the privileges it needs for its job

IAM: identity and access management

- Policies and technology that make sure the right users get the right access
- **Identify, authenticate, authorize**
- IAM puts least privilege into practice

IAM components

Component	Meaning
Logical organization	Boundary or domain for users and processes
Users	Have own credentials, no permission by default
Groups	Collections of users, no credentials
Policies	Collections of permissions that allow or deny
Roles	Collections of policies, assumed for temporary credentials (RBAC)

Multi-factor authentication

Two or more different factors

Factor	Example
Something you have	Token, smart card, phone
Something you know	Password, PIN
Something you are	Fingerprint, face, voice, iris
Somewhere you are	GPS location

Federated identity and SSO

- **Federated identity**: one identity linked across several identity systems
- **SSO**: log in once, use many systems
- **Enterprise federation**: company directory
- **Web identity federation**: Google, Facebook login

Encryption

Symmetric
one shared key
fast
key sharing is the problem

Asymmetric
public key + private key
slower
solves key sharing

Hashing
one way, fixed length
integrity checks

Protecting data

Client side encryption: you encrypt before upload. Server side: the provider encrypts

State	Protect with
Data in use	Access control, secure memory
Data in transit	TLS / HTTPS, VPN
Data at rest	Disk and object encryption

Cloud security risks and threats

- Poor access management
- Data breach, leak or loss
- Misconfiguration
- Insecure APIs
- Account hijacking
- Lack of visibility
- DoS and DDoS attacks

Cloud monitoring

- Observing and managing operational workflow in the cloud
- Checks **availability and performance** of sites, servers, apps
- Services: **Amazon CloudWatch, Azure Monitor**, Datadog, AppDynamics

Metrics, events, logs, alerts

Item	Meaning
Metrics	Raw numbers: CPU, memory, disk
Events	Something happened: what, where, when
Logs	What systems have done, key for troubleshooting
Alerts	Reactive: threshold, anomaly or heartbeat alerts that notify or run an action

Lecture 7: Database and Analytics Services

IT4090 Cloud Computing

- SQL and NoSQL
- OLTP and OLAP
- Caching
- Data warehouse and data lake

Relational vs non-relational

ACID: Atomicity, Consistency, Isolation, Durability

	Relational (SQL)	Non-relational (NoSQL)
Structure	Tables, fixed schema	Flexible, document or key based
Rules	**ACID**	Rapidly changing designs
Examples	MySQL, PostgreSQL, SQL Server, Oracle, RDS	MongoDB, Cassandra, DynamoDB, Redis, CosmosDB

OLTP vs OLAP

	OLTP	OLAP
Focus	Operational data, transactions	Historical data, analysis and reports
Queries	Small and simple	Large and complex
Response	Milliseconds	Seconds to hours
Design	Normalized	Denormalized (warehouses)

NoSQL types

Type	Use case	Example
Key-value	Session data, shopping cart	Redis, DynamoDB
Document	E-commerce, content	MongoDB, CosmosDB
Columnar	Analytics, blogging platforms	Cassandra
Graph	Relationships: social networks	Neptune, Neo4j

CAP theorem again

- Pick 2 of **Consistency, Availability, Partition tolerance**
- During a partition you choose consistency or availability
- Example: MongoDB leans CP, Cassandra and DynamoDB lean AP

Caching

- Keeps **frequently requested data in memory**
- Faster access and **less load on the database**
- Two systems: **Redis** and **Memcached**

Redis vs Memcached

Redis	Memcached
In-memory key-value store, many data structures Persistent: survives reboot Replicas, backup, high availability	In-memory object store, simple Not persistent Multithreaded, scales out

Data warehouse

- Data management system for **business intelligence** and analytics
- **Structured, filtered, processed** historical data from many sources
- **ETL**: extract, transform, load via a staging area
- **ELT**: extract, load, then transform in the destination

Data lake

- Storage repository for **raw data in its native format**
- **Flat architecture**, not folders and files
- Purpose of the data is not decided yet
- Used by data scientists

Data warehouse vs data lake

	Warehouse	Lake
Data	Processed	Raw
Use	Current, known purpose	Purpose not decided
Users	Business professionals	Data scientists

Real-time processing and BI

- **Real-time (stream) processing**: process data as it arrives, continuous input and output
- Examples: fraud detection, live dashboards
- **Business intelligence**: dashboards and reports for decisions

Lecture 8: Cloud Migration

IT4090 Cloud Computing

- Migration process
- Migration strategies: the 6 Rs
- Challenges and solutions

What is cloud migration?

- Moving a meaningful part of your organization's **existing IT assets to the cloud**

Migration process: 4 phases

- **1 Opportunity evaluation**: what is the business case or trigger?
- **2 Portfolio discovery and planning**: what do we have, what depends on what, what first, how
- **3 Design, migrate, validate**: one application at a time, simplest first
- **4 Operate**: DevOps, NoOps, FinOps, SecOps

Opportunity evaluation: triggers

- Hardware renewal
- Data center lease ends
- More productivity
- Global expansion
- Mergers and acquisitions
- Standardized architectures
- Faster time to market

Discovery, planning and tools

- What is in the environment? What depends on what?
- What do we migrate first? How?
- Tools: **AWS Application Discovery Service, Azure Migrate**, third-party tools

Design, migrate and validate

- Start with the **least complex** applications
- Test, then decommission old systems
- Run **parallel environments** if needed
- Move traffic in steps, then **full cutover**

Migration strategies compared

Gartner 5 Rs	Azure 4 Rs	AWS 6 Rs
Rehost, Refactor, Revise, Rebuild, Replace	Rehost, Refactor, Re-architect, Rebuild	**Rehost, Re-platform, Repurchase, Refactor, Retire, Retain**

AWS 6 Rs

R	Meaning	Example
Rehost	Lift and shift, no changes	VM to EC2
Re-platform	Lift, tinker and shift: small changes	Database to RDS, VMs to App Service
Repurchase	Move to SaaS	CRM to Salesforce, HR to Workday
Refactor	Re-architect with cloud-native features, most expensive	Monolith to microservices
Retire	Switch off what is not needed	Unused tool
Retain	Keep for now, revisit later	Recent upgrade, low priority

Migration challenges

Challenge	Solution
Cost of migration	Plan well, move step by step, use hybrid cloud
Resistance to adapt	Leadership buy-in, intuitive tools, vendor help
Shortage of skills	Training, learning culture, vendors help skill up staff

Lecture 9: Cloud Native Applications

IT4090 Cloud Computing

- Monolith vs microservices
- Cloud native and 12-factor app
- DevOps, CI and CD
- Application modernization

Monolithic architecture

One single, indivisible unit: UI + server + database + middleware

Strengths	Weaknesses
Few cross-cutting concerns Easier debugging and testing Simple to develop and deploy	Complex to understand as it grows Hard to change Cannot scale parts separately Hard to adopt new technology

Microservices architecture

- App split into **small independent services**
- Each is deployed independently and talks through **APIs**
- Each can be updated and **scaled on its own**

Monolith vs microservices

	Monolith	Microservices
Scaling	Clone the whole app	Scale only the busy service
Deployment	All at once	Per service
Technology	One stack	Free choice per service
Weakness	Hard to change	Distributed complexity, harder to debug

Serverless and FaaS

Frequently asked in past papers

- **Serverless**: provider manages servers, you pay per execution
- **FaaS**: small functions run on events (AWS Lambda, Azure Functions)
- Event-driven, auto scaling, stateless
- Limits: cold start, run time limit, lock-in

Cloud native computing

- Apps that use the **flexibility, scalability and resilience** of the cloud
- Pioneered by born-in-the-cloud firms: Netflix, Spotify, Uber, Airbnb
- **CNCF** promotes it, focusing on containers

12-factor app (1 to 6)

Factor	Rule
1 Codebase	One codebase, many deployments
2 Dependencies	Declare all dependencies
3 Config	Store config in environment variables
4 Backing services	Treat as attached resources
5 Build, release, run	Three separate stages
6 Processes	Stateless processes

12-factor app (7 to 12)

Factor	Rule
7 Port binding	Service exposes itself on a port
8 Concurrency	Scale out with more processes
9 Disposability	Fast start, graceful stop
10 Dev/prod parity	Keep environments alike
11 Logs	Treat logs as event streams
12 Admin processes	Keep admin tasks in source control

Cloud native tools

- **Containers** package apps
- **Microservices** split the business functions
- **Service mesh** manages service-to-service communication
- **Orchestration** (Kubernetes) provisions, deploys, operates and scales containers

Cloud native practices

- **DevOps**: development and operations working together
- **SRE**: Google's reliability practices
- **NoOps**: everything automated, no manual operations
- **CI** and **CD**

CI, delivery and deployment

Tools: Jenkins, GitLab, GitHub, CircleCI · AWS CodePipeline, CodeBuild, CodeDeploy · Azure DevOps

Practice	Meaning
Continuous integration	Merge code often, automatic build and tests
Continuous delivery	Package passes tests, **manual approval** to release
Continuous deployment	Passing checks go to production **automatically**

Application modernization journey

- Refactor legacy software to match today's business needs
- Traditional app
- Rehost as a container
- Add new microservices beside it
- Rearchitect parts as microservices
- Fully transformed microservices application

Lecture 10: Infrastructure as Code (IaC)

IT4090 Cloud Computing

- Why manual processes fail
- IaC, tool types, approaches
- Terraform and CloudFormation

Risks of manual processes

- No **repeatability** at scale
- No **version control**, hard to roll back
- No **audit trail** for compliance
- **Inconsistent** configurations between servers

What is IaC?

- Provision and manage cloud resources by **defining them in a template file**
- Human-readable and machine-readable
- Config that lived in a console now lives in a **repository as code**

Without IaC vs with IaC

Without IaC	With IaC
Manual setup each time	Code reused for every deployment
Slow teardown and deployment	Instant
Errors in setup	Stored in version control
Manual rollback	Simple, instant rollback

Types of IaC tools

Type	Job	Examples
Configuration management	Install and manage software on existing servers	Ansible, Puppet, Chef, SaltStack
Server templating	Reusable server templates	Docker, Packer, Vagrant
Orchestration	Manage how parts work together	Kubernetes, Docker Swarm, OpenShift
Provisioning	Create the infrastructure	Terraform, CloudFormation

Imperative vs declarative

Imperative

step-by-step commands

you say HOW

Declarative

describe the end state

the tool works out how

Terraform, CloudFormation

Benefits of IaC

- **Consistency** and reproducibility
- **Automation and speed**: build or destroy in minutes
- **Version control** with Git
- Reusable stacks for dev, test and production
- Easy clean-up: delete the stack, delete the resources

IaC best practices

- Little or no documentation: the code is the documentation
- Version control all configuration files
- Test configurations constantly
- Use a modular approach

Terraform

- HashiCorp tool for **cloud and on-prem** resources
- Works with many providers through **APIs**
- Core workflow: **Write, Plan, Apply**
- **Plan** shows what will be created, changed or destroyed. **Apply** does it in the right order

AWS CloudFormation

- Describe all AWS resources in a **template** (JSON or YAML)
- CloudFormation provisions them and handles dependencies
- Resources created together form a **stack**
- AWS only, while Terraform is multi-cloud

Lecture 11: Containerization and Docker

IT4090 Cloud Computing

- Why containers
- Docker components
- Container lifecycle and commands

Why containers?

- Apps depend on libraries and OS versions, so they break between dev, test, QA, staging and production (the matrix from hell)
- A **container** bundles the app with all its dependencies
- It runs the same on a laptop, a server or the cloud

VM vs container

	VM	Container
OS	Own guest OS	Shares host kernel
Size	GBs	MBs
Start	Minutes	Seconds
Managed by	Hypervisor	Container engine

Advantages of containers

- **Portability**
- **Consistency** across environments
- **Lightweight**
- **Scalability** for microservices
- **Efficiency**, fast start

Docker

- Tool to run apps in **containers** on a Linux host
- Written in Go, uses Linux **namespaces** for isolation
- Client-server design: the client talks to the daemon over a REST API

Docker components

Component	Role
Docker daemon (dockerd)	Manages images, containers, networks, volumes
Docker client (docker)	Sends your commands to the daemon
Docker Desktop	Easy install: daemon, client, Compose, Kubernetes

Docker objects

Object	Meaning
Image	Read-only template, the blueprint
Container	Running instance of an image
Dockerfile	Text file of commands that builds an image
Registry	Store for images: Docker Hub, Amazon ECR, Azure Container Registry

Container lifecycle commands

- `docker pull IMAGE` : download an image
- `docker create` : make a container, not started
- `docker start` : start it
- `docker run` = pull + create + start
- `docker ps` running · `docker ps -a` all

Managing containers

- `docker logs` : see output
- `docker attach` : connect your terminal
- `docker exec -it ID cmd` : run a new command inside
- `docker pause` / `unpause` : SIGSTOP / SIGCONT

Stopping and removing

- `docker stop` : graceful, **SIGTERM**
- `docker kill` : forced, **SIGKILL**
- `docker restart`
- `docker rm` removes a container, `docker rmi` removes an image, `docker system prune` clears stopped containers

Lecture 12: Container Orchestration

IT4090 Cloud Computing

- What orchestration is
- How it works
- Advantages and tools

What is container orchestration?

- Automating **deployment, management, scaling and networking** of containers through their whole lifecycle
- Lets software run consistently at scale across many environments

Why do containers need orchestration?

- A few containers can be managed by hand, **hundreds cannot**
- Containerized microservices are the compute units of cloud native apps
- A CI/CD or DevOps pipeline needs it

How it works

- A **cluster** is a group of nodes across machines
- **Worker nodes** run containers
- **Control plane nodes** act as the orchestrator
- Managed through APIs and GUIs

The three-step cycle

- **Provisioning**
- **Deployment**
- **Lifecycle management**
- Repeats as part of an agile or DevOps pipeline

Advantages

Area	Benefit
Automation	Deploying, scaling, managing is automatic and faster
Reliability	Health checks and self-healing reduce downtime
Cost and efficiency	Smart scheduling, scale up and down, pay for what you use
Security and portability	Less human error, runs on-premises or any cloud, avoids lock-in
Modern practice	Fits DevOps, CI/CD and microservices

Tools

- **Kubernetes** (EKS, AKS, GKE are managed versions)
- **Docker Swarm**
- **Red Hat OpenShift**
- Amazon ECS

Kubernetes words

Term	Meaning
Cluster	All the nodes
Node	A machine that runs containers
Pod	Smallest unit: one or more containers
Service	Stable address and load balancing for pods